

Event Dispatch Pipeline (EDP)

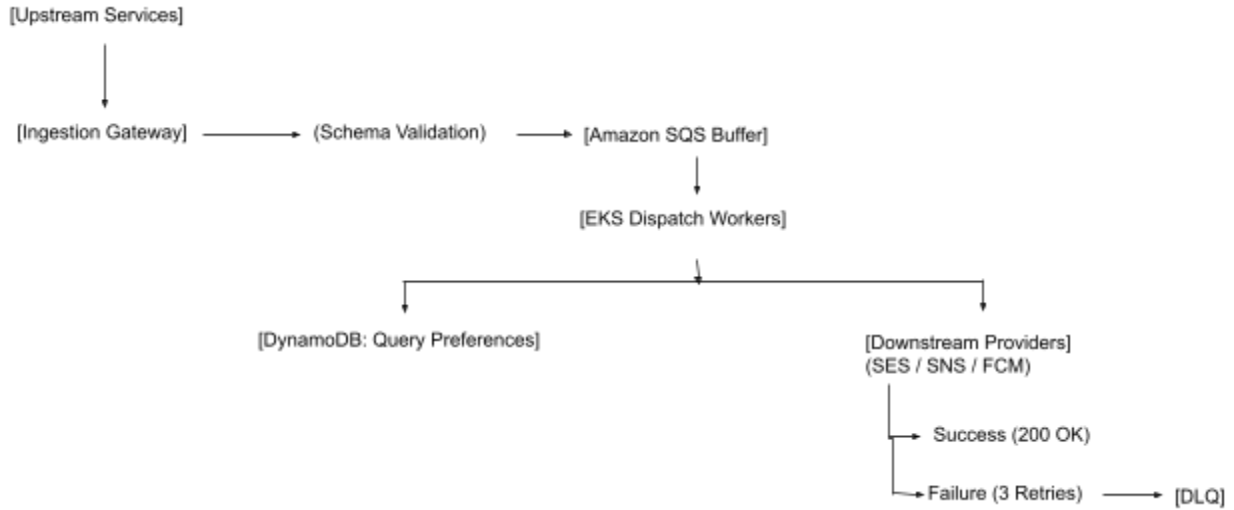
Architecture Overview

The Event Dispatch Pipeline (EDP) is a distributed, asynchronous messaging system that routes transactional notifications from upstream domain services to end-user communication channels. EDP decouples message producers from delivery providers, ensuring fault isolation, sub-200 ms end-to-end delivery latency, and support for up to 15,000 transactions per second (TPS) at peak load.

Core Architecture Components

Component	Technology	Responsibility
Ingestion Gateway	HTTP / REST API	Authenticates incoming requests, validates JSON schemas, and returns an immediate acknowledgment to prevent upstream blocking.
Message Buffer	Amazon SQS	Buffers incoming message payloads to absorb traffic spikes and isolate downstream workers from traffic bursts.
Dispatch Workers	Go on Amazon EKS	Polls the buffer in batches (up to 10 messages), resolves recipient preferences, and dispatches payloads to channel providers.
Routing & State Store	Amazon DynamoDB	Maintains user subscription configurations and delivery channel preferences.
Delivery Providers	SES, SNS, FCM	Delivers finalized payloads across email, SMS, and mobile push channels.
Dead-Letter Queue (DLQ)	Amazon SQS DLQ	Retains messages that fail delivery after three retry attempts for audit and manual replay.

End-to-End Message Flow



1. **Ingestion & Validation:** Upstream services send an HTTP POST request containing a JSON payload to the Ingestion Gateway. The gateway validates payload syntax and immediately returns an HTTP 202 Accepted response.
2. **Buffering:** Validated messages are enqueued onto the primary Amazon SQS buffer queue.
3. **Batch Processing:** Dispatch workers on Amazon EKS poll the queue in batches of up to 10 messages.
4. **Preference Resolution:** Workers query DynamoDB to retrieve the recipient's active communication preferences (such as opted-in channels and quiet hours).
5. **Channel Dispatch:** Workers format the payload for the target provider (Amazon SES for email, Amazon SNS for SMS, or Firebase Cloud Messaging for push notifications) and initiate delivery.

Fault Tolerance & Error Handling

- **Upstream Isolation:** Asynchronous SQS queuing prevents downstream provider latency or outages from impacting upstream service availability.
- **Transient Failure Retries:** If a downstream delivery provider experiences throttling or network timeouts, the worker retries the request up to three times using exponential backoff.
- **Dead-Letter Handling:** Payloads that fail all three retry attempts are automatically moved to a Dead-Letter Queue (DLQ). This preserves message state for investigation and reprocessing without blocking the primary queue.